



ISLAMIC UNIVERSITY OF TECHNOLOGY



Phase A&B (SW) Report

EEE 4308

Digital Electronics Lab Project

Project Name:

Digital Snake Ladder Game

Group Name:

NSB

Group Members:

	Student ID	Member Name
1.	220021346	Musfique Ullah Khan Siam
2.	220021348	Fasahat Labib
3.	220021332	Fardin Ahmed Chowdhury
4.	220021345	MD.Mahir Rahman
5.	220021344	Kazi Mokabbir Hossain
6.	220021327	Kazi Adit Raiyan

Supervisor:

Mr. A. K. M. Rakib

Lecturer, Department of Electrical and Electronic Engineering,
Islamic University of Technology.

Objective:

The goal of this project is to implement the snake ladder game digitally on both software and hardware using combinational and sequential logic circuits.

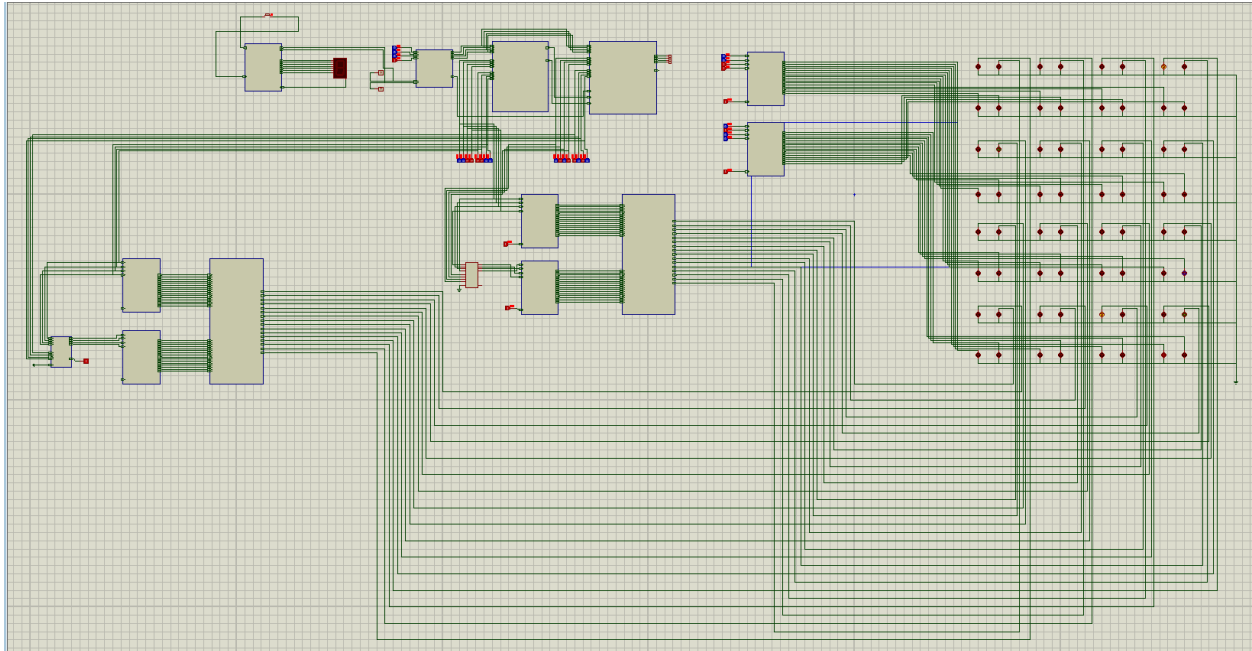
Brief Description of the Project:

The snake ladder game is a classic board game played between two or more players. This is a game of chance means the outcome of the game is a random event determined by the roll of a dice. The goal of this game is to be the first to reach the ending square, usually 100, from the beginning square, 1. While snakes “eat” the player if met and send them back to a lower square, and ladders promote the player to a higher square.

This project aims to design the game on a 4x4 LED grid. There will be a total of 16 positions, denoted with 0 through 15 in logic. Each position is visually represented by 4 LEDs. The first two for the players and the latter two for the snake and the ladder. We have chosen red and blue for player 1 and 2 respectively, yellow for the snake, and green for the ladder. A roll of dice, simulated by the dice module to generate a pseudo-random number from 1 through 3 shifts the players’ position by that number. If a snake or a ladder is met, the player is demoted or promoted. The new 4-bit position of the player is stored in the memory and a player selector module is used to make players take turns to roll the dice. The player reaching the position 15 first wins the game. A win is determined by the winning module that compares the player’s current position to the winning condition. The win is then visually represented with buzzers and LEDs to notify the other player about the game result. There is also a reset module that can start the game all over from any position.

Circuit Diagram:

Here is the circuit prepared upon completion of phases A&B. This includes the board module for visual representation of the players, the snake, and the ladder, the decoder modules for snake position and ladder position, the player position decoder, the snake and ladder logic module, and the dice module which was a task for phase C.

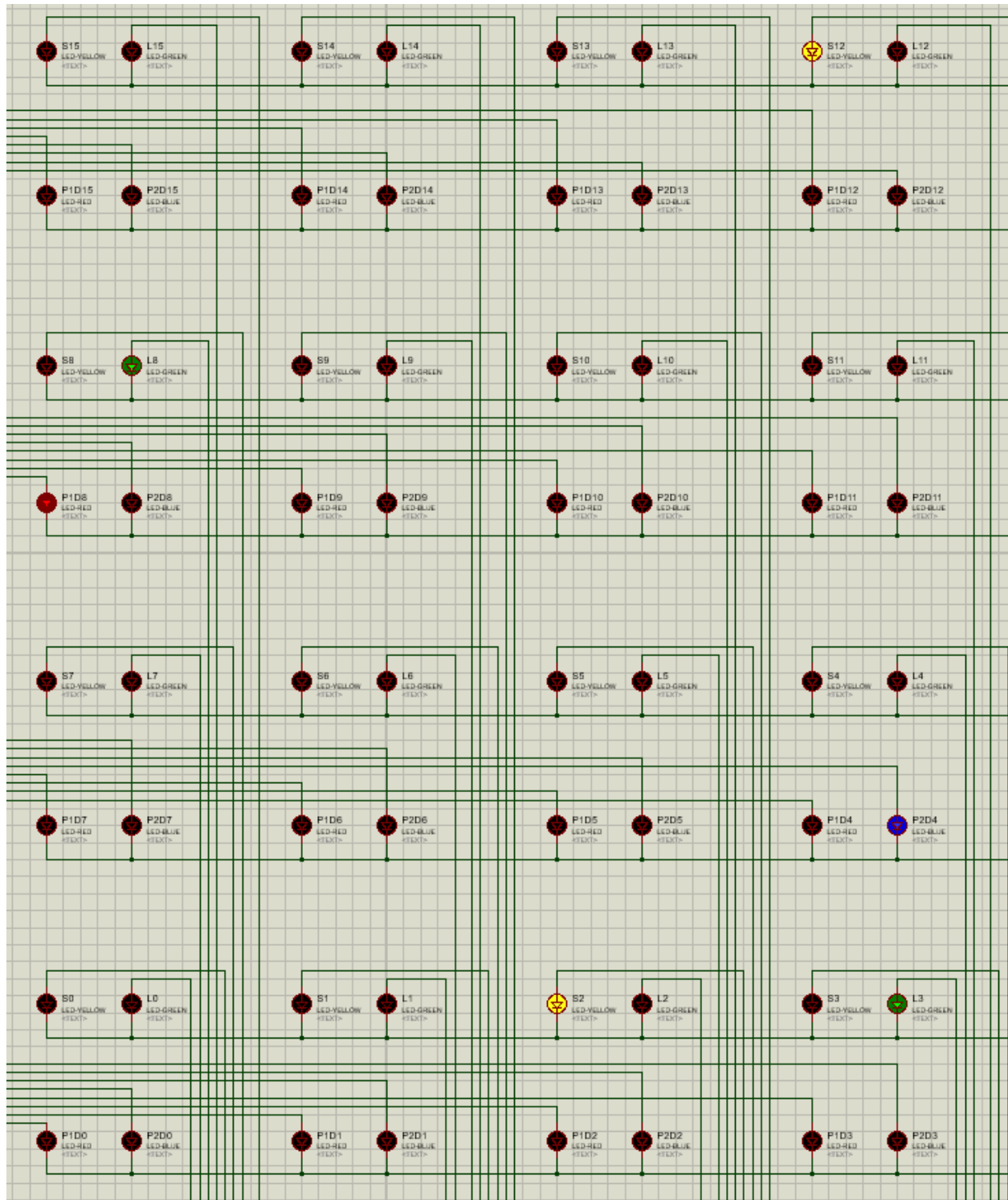


In the next segments, we have discussed each of the modules separately, the logic behind them, and ICs and components that were involved in assembling each of the modules.

LED Board Module:

The board is designed with a total of 64 LEDs with 4 LEDs making a position grid. This gives a total of 16 positions from 0 through 15. We have picked the colored LEDs available in the Proteus local library to make the board. All the red LEDs are connected to the player 1 position decoder, the blue LEDs are connected to the player 2 position decoder. The yellow and green LEDs are connected to the snake position decoder and the ladder

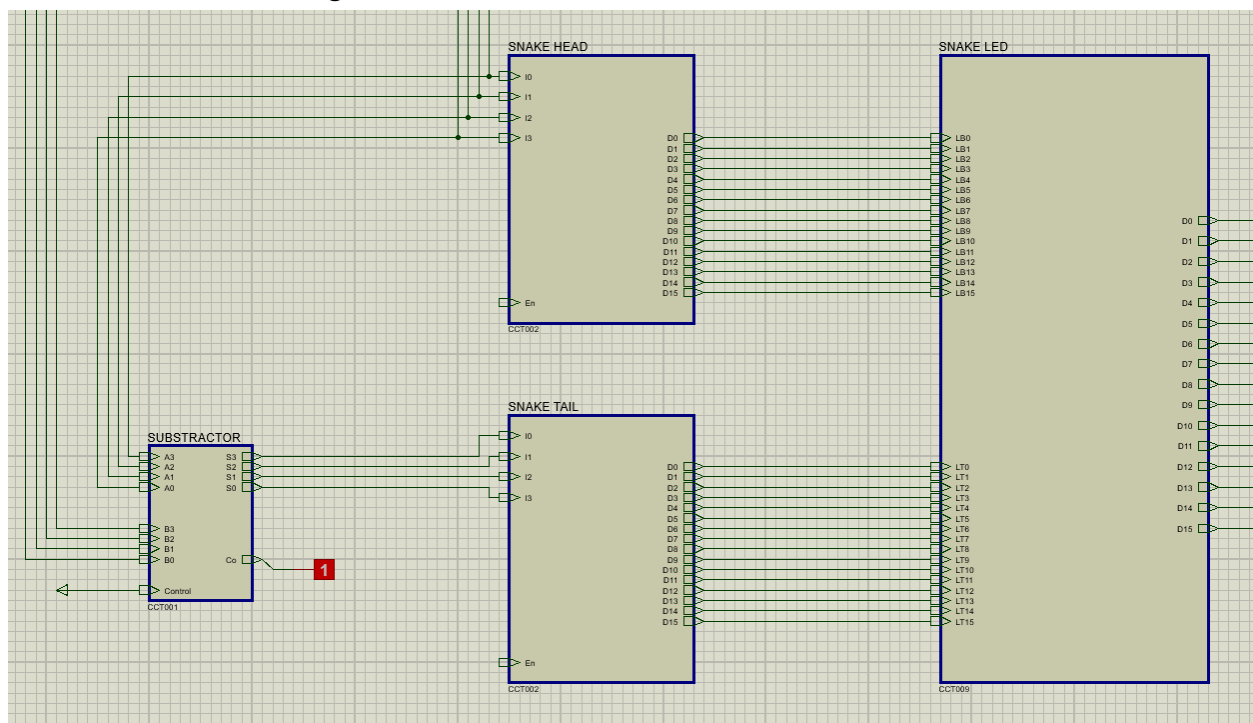
position decoders respectively.



Also seen in the circuit is that the other end of the LEDs are shorted and connected to ground together.

Snake Position Decoder:

The snake position decoder determines the position of the snake on the board. On our circuit the snake head is on square 12 and the tail is on position 2. Meaning the snake has a length of 10. The yellow LEDs on squares 12 and 2 are statically lit to imply the fixed position of the snake on the board. Whenever players reach square 12 they are demoted to square 2. This is done using a 4-bit subtractor.

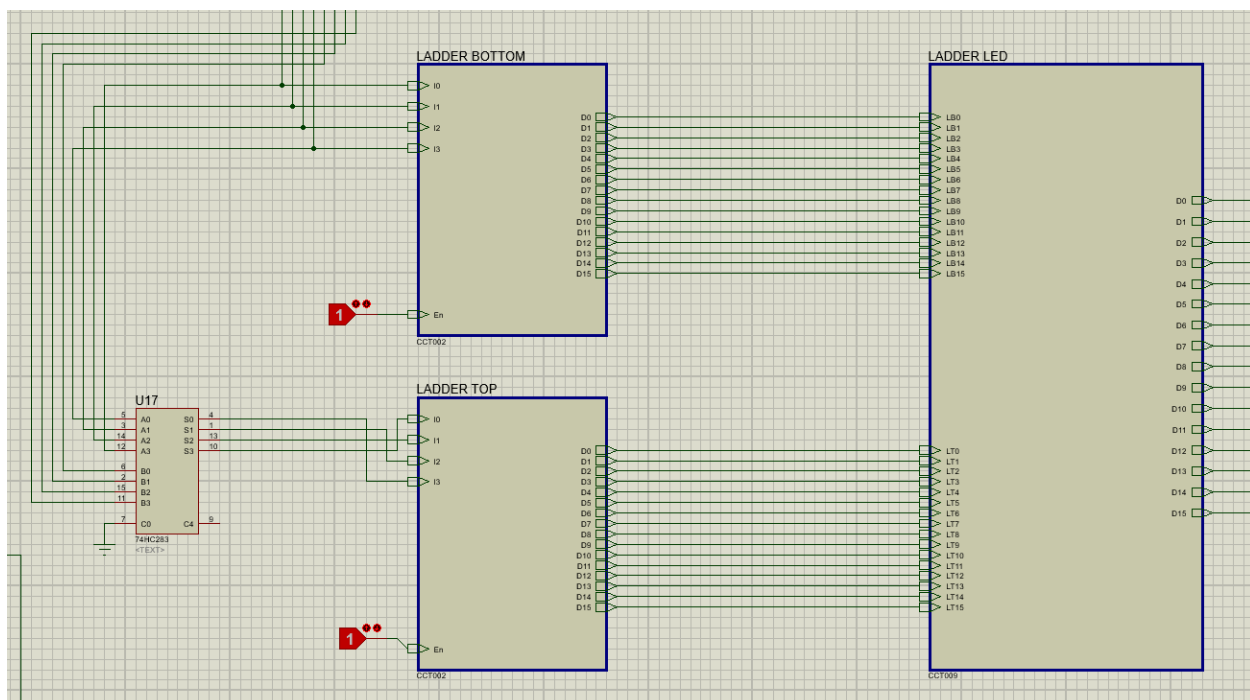


The snake head subcircuit and the 4 bit subtractor take inputs from the preset values of the snake head (1100) and snake length (1010) respectively. The snake head value is then put into the subtractor to perform the subtraction operation. The subtractor is originally an IC-74HC283 4-bit binary full adder that is given the 2's complement of the snake length and is added to the snake head value to perform subtraction using addition. The subtraction gives the value of the snake tail, the square the player who encounters the snake is demoted to, which in this case is 0010. The snake head and snake tail outputs are driven into snake led where every alternating pin (e.g. D0 of snake head and tail) are connected with an XOR gate, from here every output is connected with the leds sequentially. Such approach is done to remove the conflict between two

decoders while turning on two LEDs at once. Each decoder wants to turn on a specific LED and turn off the rest, that would rise a conflict when we want to turn on two LEDs at once, but including the corresponding outputs of two decoders by XOR, such output will be ultimately 1. When both decoders want to lit up a specific LED, (1,1 input in a XOR gate), that means snake / ladder ends at one place, the XOR output will be 0. No LED will turn on that place indicating an error has occurred.

Ladder Position Decoder:

Like the snake position decoder, the ladder position decoder is used to visually represent the fixed position of the ladder on the board. The ladder bottom position is set at 0011 and the ladder has a length of 0101. This gives the ladder top position of 1000. Thus a player reaching square 3 is automatically promoted to square 8 via the ladder. This operation is done using a 4 bit binary full adder. Also just like the snake the ladder bottom and ladder top LEDs are statically lit to show the fixed position of the ladder on the board.

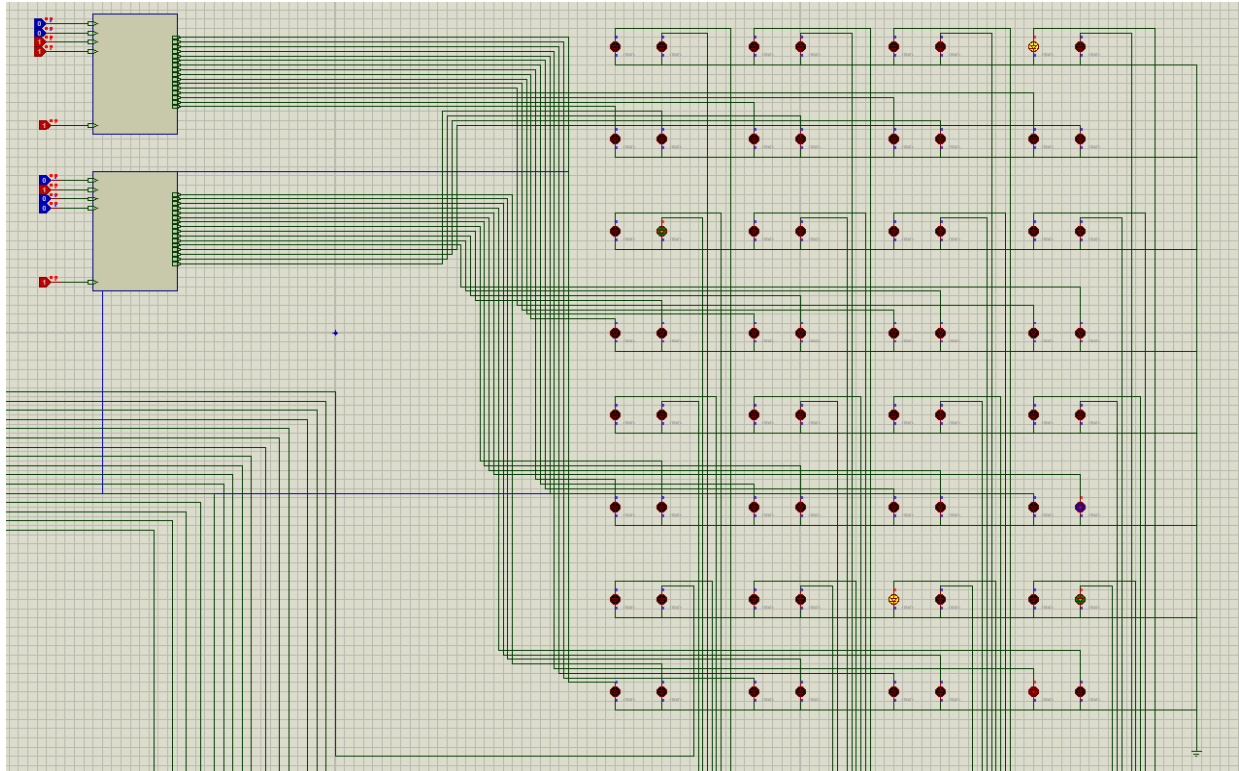


The ladder bottom and the ladder length has preset values just like the snake case. Here again we use an IC-74HC283 4-bit binary full adder to add the numbers to determine the position of the ladder top. The addition operation gives the value of ladder top, in this case 1000, the square a player is promoted to once they encounter the ladder bottom. Here to light up the two LEDs for determining the position of the ladder we use a similar approach like we did with snake LED, i.e. we use XOR gates with the alternating ladder bottom and top pins, and the outputs are connected with the LEDs sequentially.

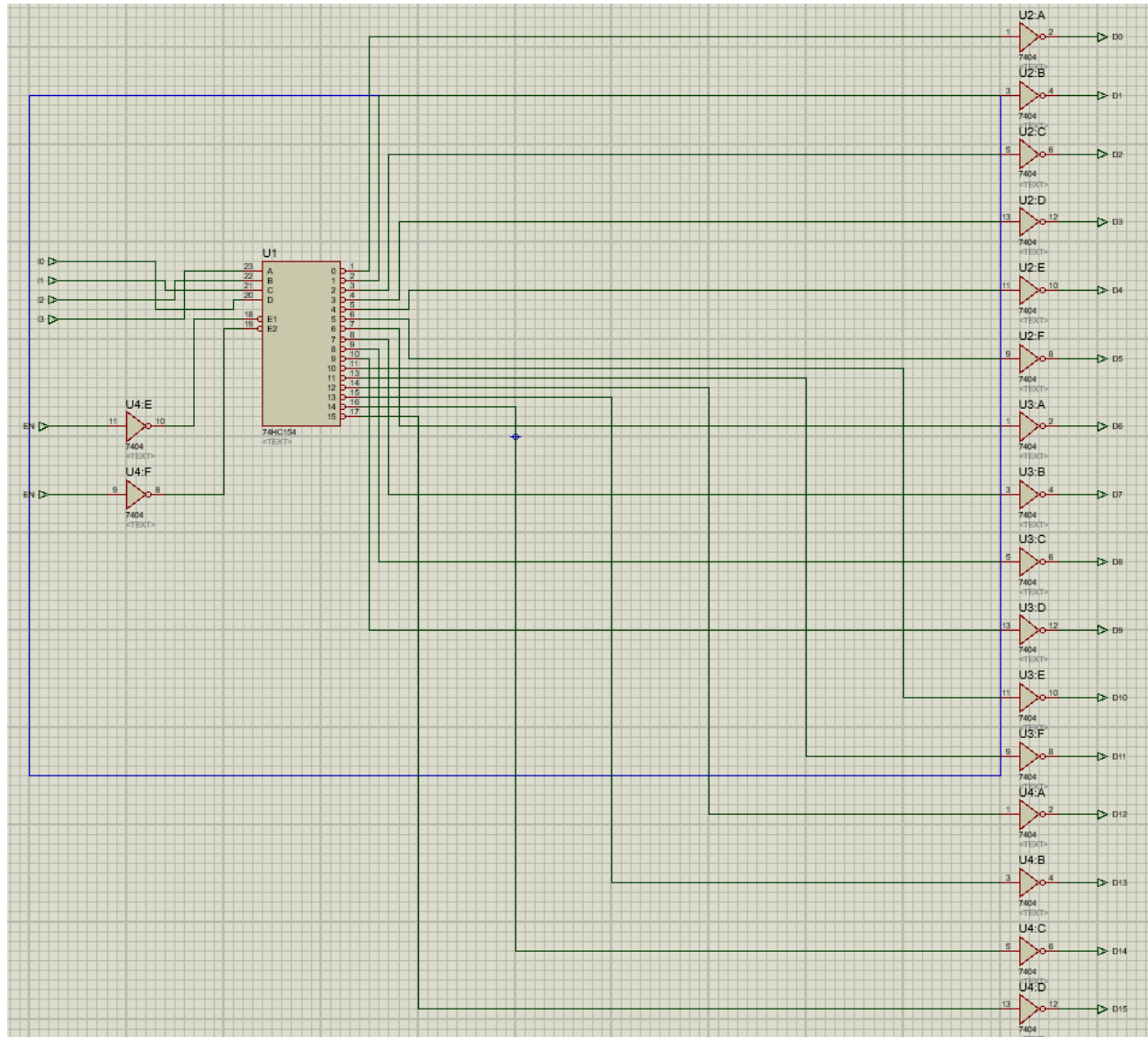
There is a significance of using an adder to determine the final positions via the snake and the ladder instead of using fixed values for the final positions. This allows us to tweak the values of ladder and snake lengths as well the start positions of them to spice the game a little bit. The adder will automatically get the final value so we can set start values and the snake or ladder length to any value of our choice to modify the game later.

Player Position Decoder:

There are two decoders to show the players' positions on the LED grid. The player position decoders are supposed to be connected to the memory where the previous values of their positions will be stored. The memory is fed by the dice module to determine the next position of the players. The 16 outputs of the player 1 decoder and player 2 decoder are connected to all 16 of the red and blue LEDs respectively.



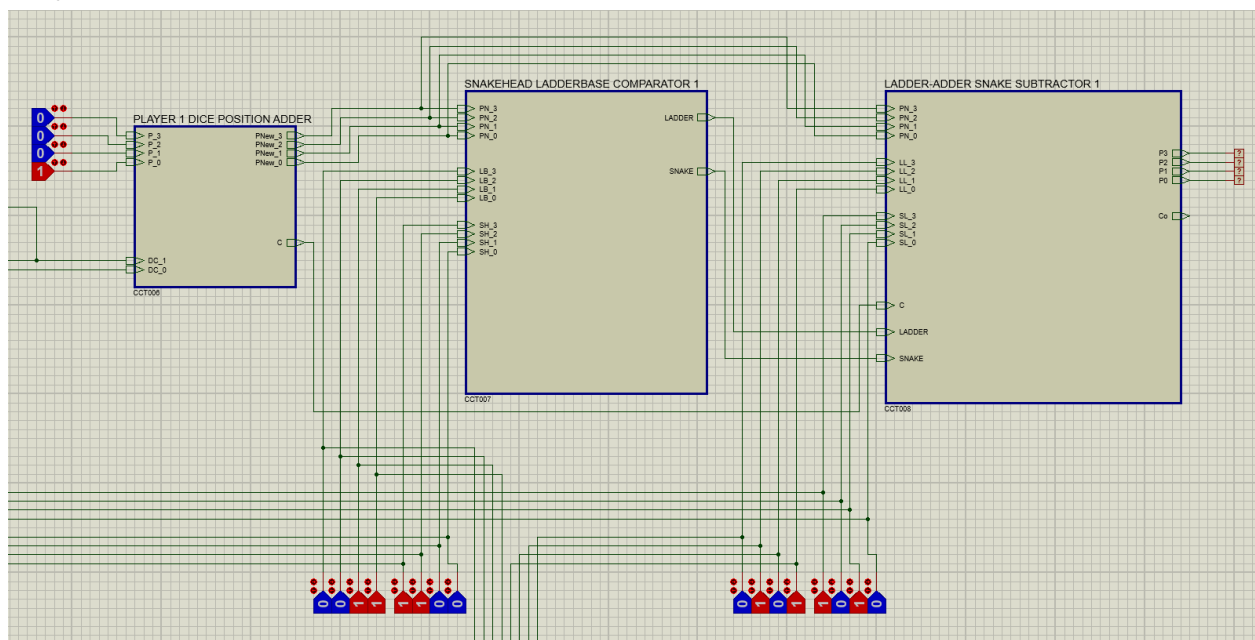
However, for phases A&B, we were not required to design the memory and reset modules. So we have connected the inputs of the decoders to the logicstates instead. The value on the logicstates now directly give outputs on the LED grid. For demonstration, we have put player 1 on position 3 and player 2 on position 4. The player position decoders are designed using two IC-74HC154 4-to-16 line decoders. A more detailed description of decoder using the IC can be seen in the following figure:



This is the child sheet of the player position decoders with the IC. The input pins of the 74HC154 receive the input from the logicstates. We have used 7404 hex inverters on the Enable pins as well as the output pins because this IC by default gives a negative output that is denoted by the bubble. The output is fed to subcircuit terminals which go to the LED grid to show players' positions on the board.

Snake and Ladder Logic Module:

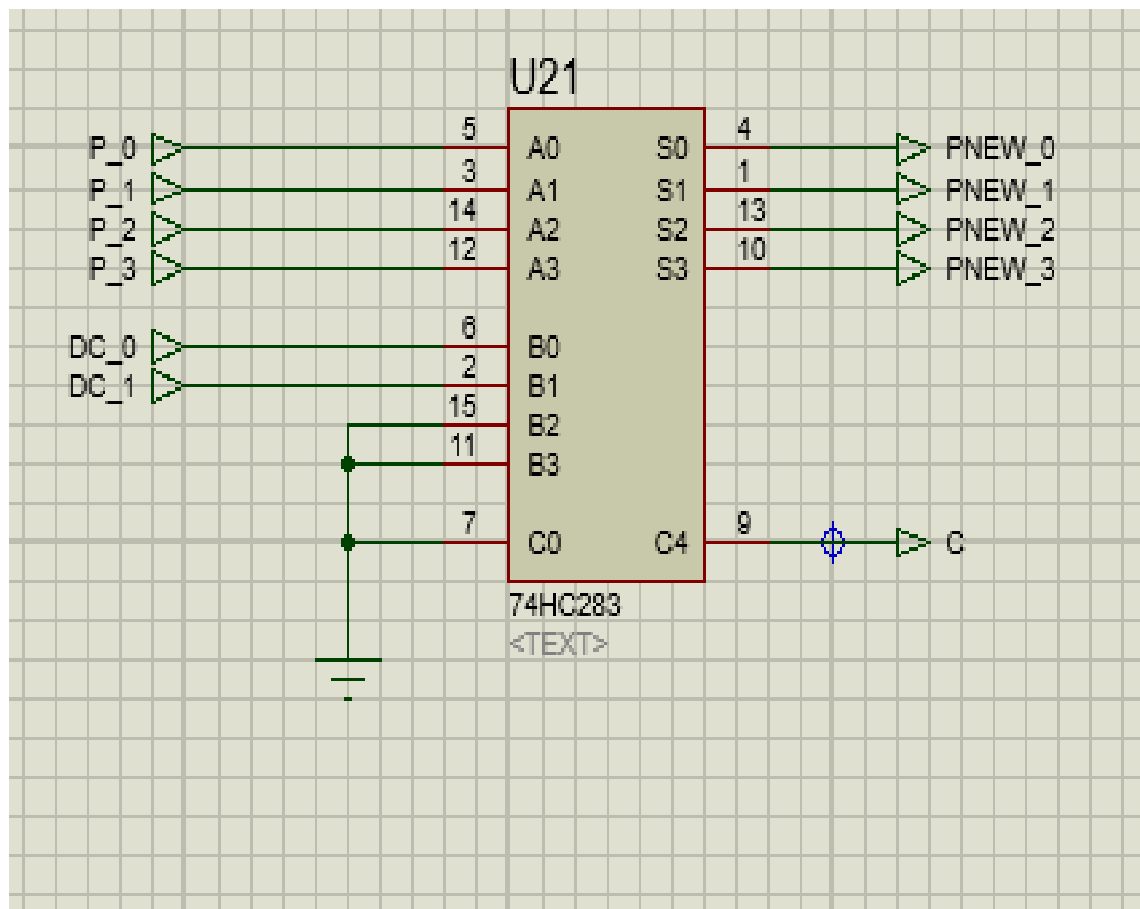
The snake and ladder logic module determines the current player's next 4 bit location depending on the dice value after checking if a snake or a ladder has been encountered. This module takes as input the current position, the dice value, and the locations of snakes and ladders on the board. The current position of the player will be taken from the memory module. The dice value is found from the 2-bit pseudo-random number generator. The snake and the ladder positions are preset values. This logic module first adds the dice value to the current player position to get a primary value for the player's next position. Then comparators are used to compare the tentative value to snake head or ladder bottom values. If a snake or ladder is met, necessary operations are performed to transfer the player's position to relevant squares.



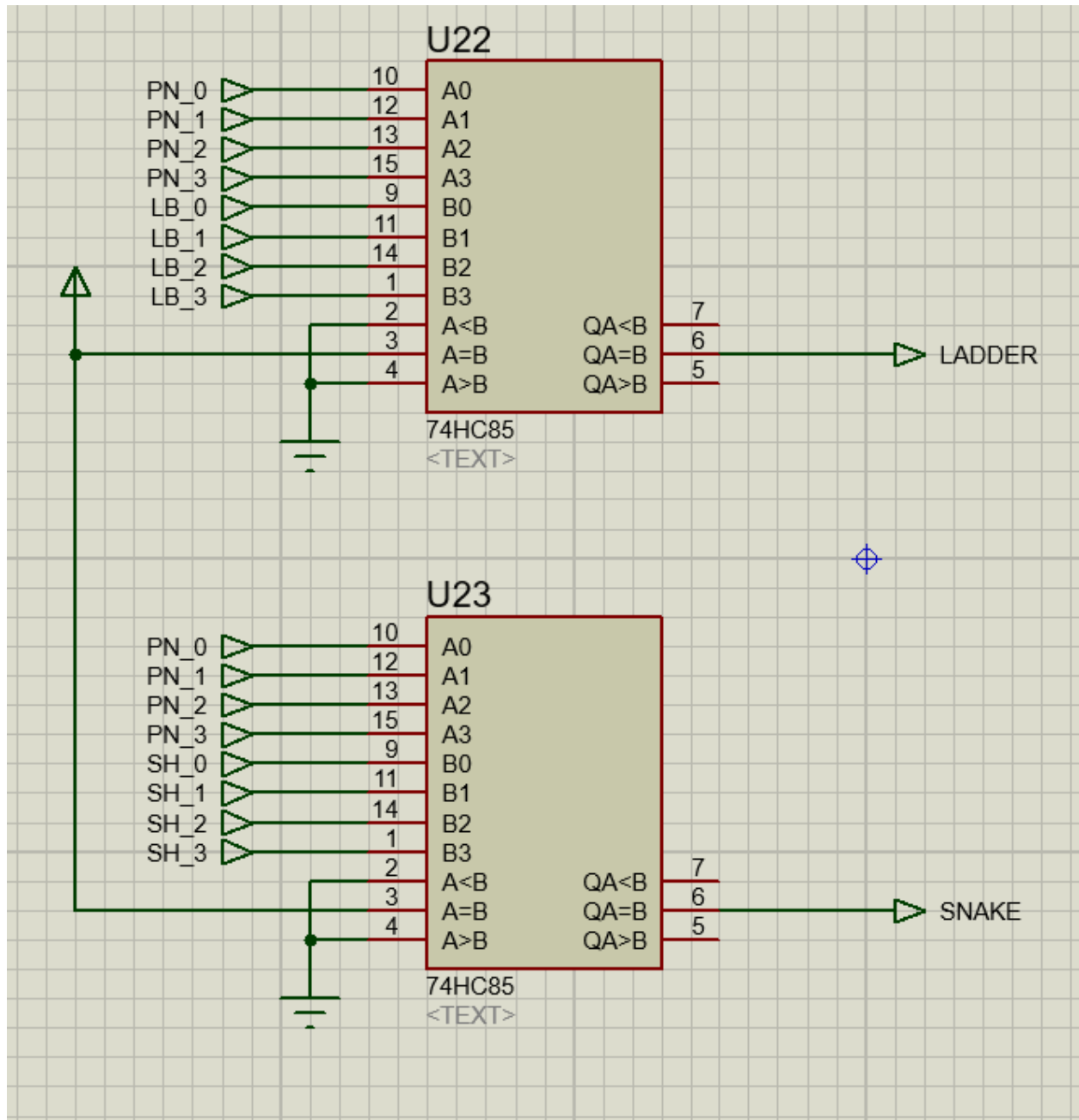
The snake ladder logic consists of three sub circuits, the "Player Dice Position Adder", the "Snakehead and Ladder Start Comparator" and lastly the "Ladder Adder and Snake Subtractor". The logic takes 4 bit position of selected player, 2 bit dice value, 4 bit ladder start, ladder length, snake head and snake length values as inputs and sends the outputs of 4 bit player's new position value and carry bit to the memory block. The player dice position adder adds the 4 bit player position value and dice value and

brings up new output with a carry. Next step is to compare if the new 4 bit value is equal to the snake head or ladder start 4 bit value by using a comparator, giving output ladder and snake. It can have three possibilities, 00/01/10 based on this the ladder adder and snake subtractor subcircuit handles three ways of calculation regarding new position of player.

The Player Dice position adder is consisted of one 4 bit parallel adder ic only (IC 7483), where 2 bits from MSB in dice were shorted to ground to make the 2 bit dice value to 4 bit dice value having two zeroes at first.

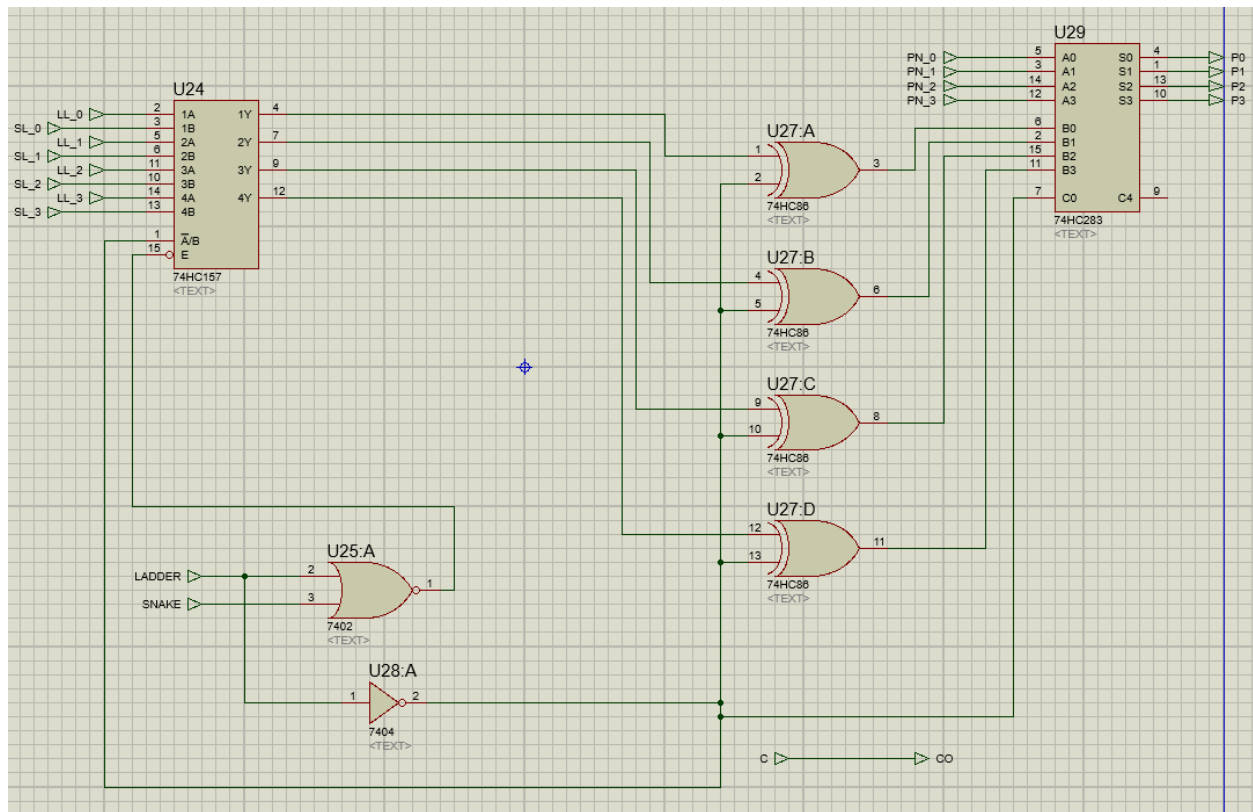


The Snakehead and Ladder Start Comparator compares this summed up 4 bit by two 4-bit comparator ICs (IC 74HC85).



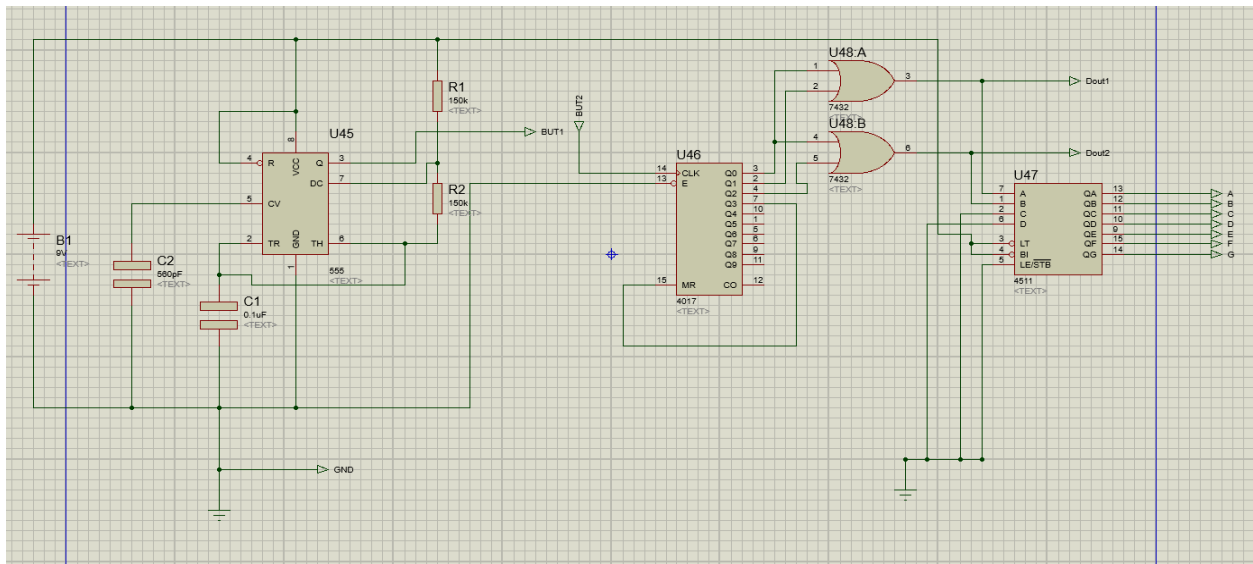
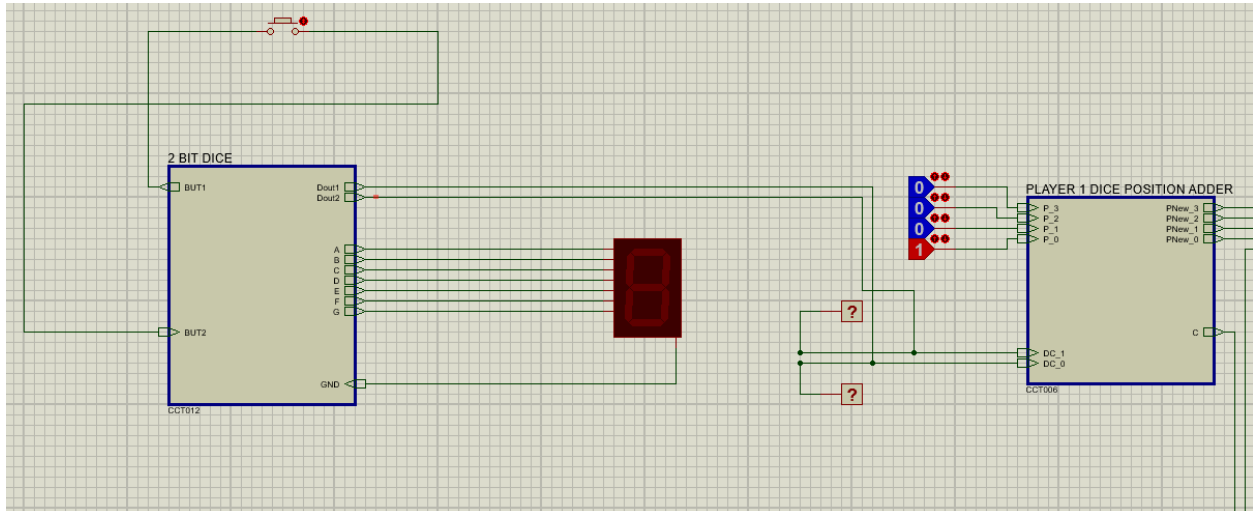
Ladder Adder and Snake Subtractor consists of 2 to 1 MUX of 4 bit, some logic gates like NOR, NOT and four XOR gates and a 4 bit parallel adder. The subtractor is made using adder and XOR gates where one end of XORs are shorted to the NOT of Ladder value, as the subtractor will turn on if ladder is 0, snake is 1. The MUX gives the values of ladder start or snake head value of 4 bit to add or subtract with the player position whenever the condition meets. If ladder and snake is 00 we expect not to add or subtract anything from the player position value so we placed a NOR gate to those

ladder and snake values and connected to the Enable pin of MUX, where 00 condition will turn off the MUX. Here IC 74HC157, 74HC86, 7404, 7402 and 7483 are used.



Dice Module:

This module generates a 2-bit pseudo-random number between 1 and 3. The Dice Module uses a high-speed counter to generate a random number between 1 and 3 and shows the result on a 7-segment display, simulating a dice roll. The quick number cycling and the player's timing while releasing the dice button provides the unpredictability factor.



The Dice Module is created using 3 components, they are a 555 timer , a clock counter and a seven segment BCD decoder and a seven segment display to see the output from the dice. There is a button connected in the middle of the 555 timer and the clock counter.

555 timer:

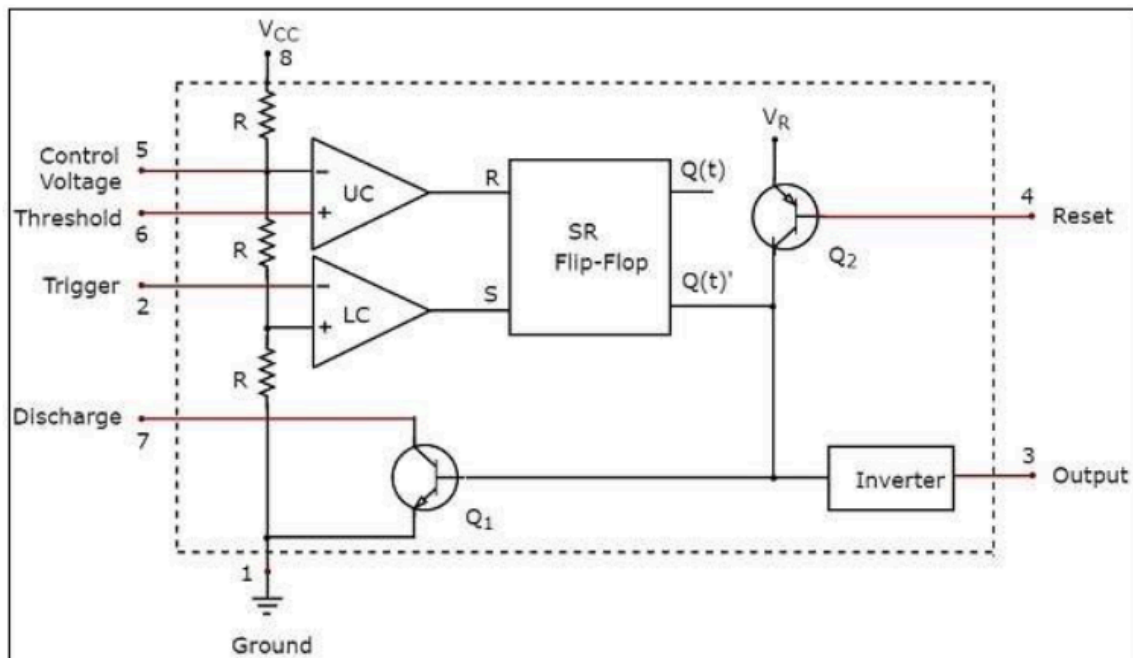
The 555 timer IC is an integral part of any electronics projects. Be it a simple project involving a single 8-bit microcontroller and some peripherals or a complex one involving a system on chips (SoCs), a 555 timer is involved. These provide time delays, as an oscillator and as a flip-flop element among other applications.

Here the 555 timer is used as a multivibrator to create a clock pulse that is fed into the clock counter. 555 timer changes the output from 0 to 1 and 1 to 0 in a fixed frequency that is dependent on the value of resistors and capacitors, here we used 150k and resistors and a capacitor with 560pF and 0.1 uF, so that we gate a high frequency clock pulse.

Functional Diagram

The pictorial representation showing the internal details of a 555 Timer is known as functional diagram.

The **functional diagram** of 555 Timer IC is shown in the following figure –



We used the following formula

Monostable

The signal duration of the output can be modeled as Time in seconds (T) equal to the constant 1.1 multiplied by the resistance R measured in ohms (Ω) multiplied by the capacitance value C measured in Farads (F).

$$T = 1.1R_1C_1$$

So, for a circuit with 10,000 (10k) ohms (Ω) of a resistance and a capacitance value of 1,000 microfarads (1000 μF), the total length of the output signal would be as follows

$$T = 1.1 \times 10,000\Omega \times 0.001F = 11s$$

Astable

The continuous output signal in Astable mode allows you to determine both the frequency in Hertz (Hz), how long it takes the signal to repeat and the duration of time the signal is in the high and low states. All three values that would define the output signal can be found using the values of R1 and R2 measured in ohms (Ω) and the capacitance value C1 measured in Farads (F).

$$T_h = 0.693(R_1 + R_2)C_1$$

$$T_l = 0.693R_2C_1$$

$$f = \frac{1.44}{(R_1 + 2R_2)C_1}$$

So, for a circuit with resistors of 10,000 (10k) and 15,000 (15k) ohms (Ω) of a resistance and a capacitance value of 10 microfarads (10 μF), the output signal would have the following characteristics

Time High

$$T_h = 0.693(10,000\Omega + 15,000\Omega)0.00001F = 0.1732s$$

Time Low

$$T_l = 0.693(15,000\Omega \times 0.00001F) = 0.1039s$$

Frequency

$$f = \frac{1.44}{(10,000\Omega + 2 \times 15,000\Omega)0.00001F} = 3.6Hz$$

Calculation of the 555 timer:

CHOOSE CONFIGURATION

Monostable

Astable

R₁ RESISTOR VALUE

150

kΩ

R₂ RESISTOR VALUE

150

kΩ

C₁ CAPACITANCE VALUE

0.1

μF

FORMULAS

$T_h = 0.693(R_1 + R_2)C_1$

$T_l = 0.693R_2C_1$

$f = \frac{1.44}{(R_1 + 2R_2)C_1}$

TIME HIGH

20.79

mS

TIME LOW

10.395

mS

FREQUENCY

32

Hz

V_{cc}

R₁

7 (DISCHARGE)

R₂

6 (THRESHOLD)

2 (TRIGGER)

C₁

1 (GND)

555

8 (VCC)

4 (RESET)

3 (OUTPUT)

V_{cc}

1 (GND)

TIME HIGH

20.79 milliseconds

TIME LOW

10.395 milliseconds

Search Catalog

Clear Selection

Clock counter:

A Counter is a device which stores (and sometimes displays) the number of times a particular event or process has occurred, often in relation to a clock signal. Counters are used in digital electronics for counting purposes, they can count specific events happening in the circuit. For example, in the UP counter a counter increases count for every rising edge of clock. Not only counting, a counter can follow a certain sequence based on our design like any random sequence 0,1,3,2... .They can also be designed with the help of flip flops. They are used as frequency dividers where the frequency of a given pulse waveform is divided. Counters are sequential circuits that count the number of pulses can be either in binary code or BCD form. The main properties of a counter are timing , sequencing , and counting. Counter works in two modes : Up and down counters.

Decade Counter:

A decade counter counts ten different states and then reset to its initial states. A simple decade counter will count from 0 to 9 but we can also make the decade counters which can go through any ten states between 0 to 15(for 4 bit counters).

In the following circuit we used a decade counter to count the values. As we only need a 2 bit dice so the value would be 1,2, or 3 (01,10,11), so we used only the first 3 pins for the count then we selected the 4th(Q3) pin to reset the counter. SO that it quickly just counts 1 to 2 to 3 and then to 1 again and it works like that.

And we connected the clock pin to the input frequency that we get from the 555 timer and connected a button in the middle.

So, The module rapidly cycles through the numbers 1 through 3, which are displayed on a 7-segment display. The dice button initiates the counter, and when the button is released, it stops and locks in the current number determined by the dice roll.

A **7-segment display** linked to a BCD to 7-segment decoder is used to show the output.

The dice button is pressed to start the counter, which quickly cycles through the numbers until the button is released, at which time the held and displayed number is revealed.

There are 2 OR gates because when the counter is at 1 (1 0 0) the output in the seven segment decoder must be (0 1) when the counter is at 2 (0 1 0) the 7 segment must show the value two (1 0) and likewise when the counter is at 3 (0 0 1) the display should show 3 (1 1) for making this circuit we used 2 OR gates as if one of the input of a or gate is 1 the output should be one . So Q0, Q1 connected with or-gate is connected to the first input of the 7-seg display and Q0 and Q2 are connected to another or gate to the second input of the 7-seg display .

Debounce circuit is not shown in this phase.

Problems Faced:

While making the board module and connecting the LEDs with their respective circuit the wires were getting tangled, so we cautiously routed every wire to easily navigate all connections.

While making the ladder and snake position decoder, at first we only used one decoder for each ladder and snake. But two LEDs were supposed to be lit for both ladder and snake position. After some calculations we used an extra decoder for both ladder and snake position. Here both of the outputs of the decoder were driven into an XOR gate and then connected with the LEDs, this solved the problem.

As our circuit got bigger it was getting hard to navigate through the components, so we used sub-circuits to make the circuit smaller visually.

Discussions:

In phases A&B, we have set up the initial yet crucial components of the digital snake ladder game successfully. The board module has given the layout of the game. The LEDs are connected and well working. We have also designed the player, snake, and ladder position decoders. Now we can visualise the positions of the snake and the ladder. We are also able to place the player LEDs on any of the squares of our choice. However, as the memory and the player selector modules are still missing, we can not yet take turns to roll the dice or to make the player LED move to another position from an existing position. Hence the game is not yet ready to be played. We have additionally designed the dice module and showed comprehensive descriptions of it. This module is working properly and we are able to generate the desired 2-bit pseudo-random number. In the upcoming phases, we will design the memory, reset, player selector, and the winner indicator modules. Then our game will be ready to play.

Contributions:

Musfique Ullah Khan Siam	: Designed the snake and ladder logic modules, snake and ladder position decoders
Fasahat Labib	: Designed the adders, subtractors, and the dice modules
Fardin Ahmed Chowdhury	: Designed the board Module
MD.Mahir Rahman	: Report writing and checking for errors
Kazi Mokabbir Hossain	: Final debugging, simplification and error checking
Kazi Adit Raiyan	: Report writing and checking for errors

Beside all of these, the main logic behind making this game up to Phase A & B was built by the combined efforts of all the group members through rigorous discussion sessions both online and offline. As it is a group project, problem analysis and most of the problem was solved as a group and all of us made a great contribution in the overall project. We are hopeful to make a nice game at the end of all phases through combined efforts.